UNIVERSIDAD POPULAR DEL CESAR

Ingeniería de Sistemas e Informática

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III · SS462 · Cuarto Semestre

Docente: Ing. Alfredo David Bautista Romero

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. Su misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ *El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.*

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── Main.java        — punto de entrada y menú interactivo
│       ├── Datos.java       — almacén global de datos del sistema
│       ├── Proceso.java     — lógica de cálculo de la factura
│       ├── Utilidades.java  — métodos auxiliares
│       └── Imprimir.java    — todo lo que se imprime en pantalla
└── README.md                — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000

4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5%: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19%: se calcula sobre el subtotal después del descuento.
- Propina 10%: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en cada archivo, con su ubicación y descripción. Usen esto como lista de verificación durante la refactorización.

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde	Descripción
1	Arrays paralelos	Líneas 11-13	Uso de tres arreglos separados (nom, p, cant) que dependen del mismo índice en lugar de crear un objeto Producto y una colección.
2	Todo public static	Líneas 11-17	Todas las variables globales son accesibles y modificables desde cualquier parte del programa, rompiendo el encapsulamiento.
3	Nombre hardcodeado	Línea 16	La cadena "El Buen Sabor" está quemada en la variable x.
4	Variable tmp sin propósito	Línea 17	Variable declarada como public static String tmp=""; que se usa sin un objetivo claro en la aplicación.

5	Sin constructor ni métodos	Todo el archivo	Es una clase anémica; solo contiene datos y ninguna lógica o comportamiento asociado.
6	Nombres de 1-3 letras	Líneas 12-17	Variables incomprensibles como p (precio), cant (cantidad), ms (mesa), est (estado), nf (número factura).

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde	Descripción
7	hacerTodo() con múltiples responsabilidades	Líneas 12-42	Calcula el subtotal, aplica descuentos, calcula IVA, aplica propinas y muta el estado global (Datos.est, Datos.tot).
8	IVA duplicado 3 veces	Líneas 22, 28, 36	La operación aux*0.19 o sub*0.19 se repite en múltiples ramas condicionales.
9	Magic numbers sin nombre	Todo el método	Uso de literales numéricos sueltos como 0.05, 50000, 0.19, 0.10, y 3 sin estar definidos como constantes explicativas.
10	Condicionales anidados 4 niveles	Líneas 19-31	Estructura profunda: if(cont>3) > if(sub>0) > if(aux>50000) lo que hace el código difícil de seguir.
11	Código comentado 'versión anterior'	Líneas 31-34	Código muerto que se dejó comentado en lugar de usar un sistema de control de versiones.
12	procesar() con 7 parámetros	Línea 43	Firma del método (double a, double b, double c, double d, double e, int f, boolean g) que supera el límite recomendado de parámetros.
13	Variable aux innecesaria	Líneas 21-29	Se utiliza solo para guardar temporalmente el subtotal con descuento en lugar de retornar o nombrar adecuadamente.
14	Comentarios que repiten el código	Líneas 15, 24	Comentarios como // multiplica precio por cantidad justo encima de sub=sub+Datos.p[i]*Datos.cant[i]; que no aportan valor.
15	Comentarios de cierre inútiles	Líneas 18, 30, 41	Anotaciones visuales como // fin while o // fin if sub>0 que intentan compensar la mala indentación.

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde	Descripción
16	calcular() con 7 parámetros	Línea 12	Variables extremadamente crípticas en los parámetros (pr, cn, dc, iv, pp, ni, ap).
17	validar() con efecto secundario oculto	Línea 27	Si cont==0, el método altera el estado global (Datos.tot=0; Datos.tmp="";) en lugar de solo devolver un boolean.

18	Nombre del restaurante hardcodeado (3ra vez)	Línea 19	Se imprime directamente "RESTAURANTE EL BUEN SABOR - calculo aplicado".
19	Bloque de ~20 líneas de código comentado	Líneas 31-42	Código viejo (calcOld y un ciclo while) abandonado dentro del método reiniciar().
20	Variables tmp y aux2 redundantes	Líneas 16, 20-21	Asignaciones innecesarias (aux2=res; return aux2; y tmp=res*iv; res=res+tmp;) que podrían simplificarse.

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	Líneas 31-82	Calcula los totales matemáticos, imprime en consola y al final actualiza variables globales (Datos.nf, Datos.est).
22	Cálculo del total duplicado	Líneas 33-47, 85-99	La lógica para calcular subtotal, IVA y propina está clonada de Proceso.hacerTodo().
23	80% del código idéntico entre métodos	Líneas 31-114	imprimirFacturaCompleta() e imprimirFacturaResumen() comparten casi el mismo código línea por línea.
24	Nombre del restaurante hardcodeado (4ta vez)	Líneas 13, 50, 102	Uso repetido de cadenas de texto de presentación en lugar de centralizarlas en variables o configuración.
25	Formato printf repetido sin constante	Líneas 69-72, 110-113	La plantilla %-27s $%,.0f%n está pegada múltiples veces a lo largo de los métodos.

3.5 Main.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde	Descripción
26	Método main de 100+ líneas		
27	Dos Scanner para el mismo System.in		
28	Variables sin uso		
29	Lógica de negocio en el main		
30	Nombre del restaurante hardcodeado (5ta vez)		

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde	Descripción
31	Sin indentación	Todos los archivos	El código carece de espacios o tabulaciones, todo está alineado al margen izquierdo, rompiendo la jerarquía visual.

32	Llaves inconsistentes	Todos los archivos	Mezcla de estilos (K&R vs. Allman): a veces en la misma línea if(){ y a veces abajo, sin un estándar.
33	Llaves de cierre apiladas	Todos los archivos	Se cierran múltiples bloques en la misma línea o adyacentes de forma ilegible (ej. sc.close();}// fin main \n } \n }).
34	Múltiples statements por línea	Proceso.java (L41), Imprimir.java (L36)	Asignaciones agrupadas horizontalmente (Datos.est=1;Datos.tot=tot;), dificultando la lectura y depuración.

Total: 34 malas prácticas identificadas y documentadas en el proyecto.

Referencia principal: Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Fuente complementaria: Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.